Malonzo, Maria Andrea V.
TN35

Midterm Documentation

This document maps the CS0045 interactive graphics requirements to features in the current program. It explains the project, identifies the required graphics concepts, records the supplied test evidence, and includes screenshots from the running application.

1. Project Proposal

Project title

Pokémon Field: An Interactive Pokémon Story and Soccer Match

Idea and purpose

I created an interactive Pokémon-themed application that takes the player through an introductory story and laboratory, lets the player choose a starter Pokémon, then presents a soccer match against Arceus. Its purpose is to demonstrate 2D OpenGL graphics and course concepts through a small interactive game rather than a static collection of shapes.

Target user

The intended users are casual players who enjoy Pokémon and short games controlled with the keyboard and mouse.

Problem and Purpose

This project explores how OpenGL concepts can be used to create an interactive 2D application. Instead of presenting graphics as a static scene, Pokémon Field combines drawn objects, image-based sprites, text, keyboard controls, mouse selection, and game states. The purpose is to demonstrate these graphics and programming concepts in a small game that users can play.

Brainstorming Summary

I wanted to make a graphics project that felt interactive and recognizable. I developed the idea into a Pokémon-themed story with character dialogue, name entry, a laboratory, and a starter choice. I then added a soccer match so that the project would include movement, collisions, scoring, animation, and win and loss states.

Initial Idea and Final Project Idea

Initial idea: Create a Pokémon-themed interactive graphics application with characters and scenes that the player could move through.

Final idea: Create Pokémon Field, an application that takes the player through Professor Oak's introduction, player and rival name entry, two laboratory scenes, and a starter selection. The player then enters a soccer match against Arceus. The match tracks scores and ends when either side reaches three goals.

Objects and expected output

The scenes include Professor Oak, player and rival characters, Bidoof, Magikarp, Caterpie, Poké Balls, laboratory shelves and a table, a tiled floor, a soccer field, goals, a ball, and Arceus. The program begins at an opening screen, advances through dialogue and naming, moves through two lab scenes and starter selection, then runs the match and shows a victory, defeat, or closing screen.

Sketch and Initial Design

The planned screens follow this sequence:

```
[Opening screen]
      ↓ Enter
[Professor dialogue]
      ↓ Enter through dialogue
[Player name] → [Rival name]
      ↓
[First laboratory]
  Player moves with WASD
  Three Poké Balls on a table
      ↓ Click a Poké Ball
[Starter selection]
  Bidoof | Magikarp | Caterpie
      ↓ Click a Pokémon
[Second laboratory]
  Player searches for the exit
      ↓ Reach exit
[Rival event]
      ↓ Enter
[Soccer match]
  Player and Arceus compete to score three goals
      ↓
[Victory or defeat]
      ↓
[Closing screen]
```

2. Interactive Application Requirements

Status: INCLUDED. The program has more than five meaningful user interactions. These controls affect progress, movement, selection, or game outcome.

Interaction	Included project behavior
Keyboard movement	W/A/S/D moves the player in the labs and on the soccer field. Direction changes the player sprite orientation in the labs.
Name entry	Typing enters player and rival names; Backspace removes a character. Enter confirms only when the current name field contains at least one character.
Mouse selection	Clicking a Poké Ball in the first lab opens the starter screen. The player can select Bidoof, Magikarp, or Caterpie; all three choices register.

Interaction	Included project behavior
Enter progression	Enter starts the story, advances dialogue, confirms a non-empty name, begins the rival match, and moves from victory to closing. The first lab remains active until a Poké Ball is clicked.
Match actions	Moving the player into the ball can kick it. The rival AI moves toward and kicks the ball.
Retry and exit	R restarts after defeat with reset scores and positions. Enter exits immediately from defeat. After victory, Enter opens the closing screen; Enter or Escape exits from closing.

3. Programming Concepts Requirements

Status: INCLUDED. The table maps each required course concept to a concrete part of this project. The course module connections are also identified.

Requirement	Status	Included in this project
C++	Included	The application is implemented in C++ in main.cpp.
OpenGL and GLUT	Included	GLUT creates the 800 by 600 window and registers callbacks; OpenGL draws the scenes.
Functions	Included	display(), keyboard(), mouse(), drawSprite(), drawLabTiles(), blockedByFurniture(), and updateField() organize behavior.
Variables	Included	State, names, sprite textures, player/ball positions, velocities, and goal totals store game data.
Conditions	Included	State selection, movement bounds, furniture collision, ball collision, goal checks, and score thresholds control outcomes.
Loops	Included	Loops generate grid lines, draw repeated bookshelf books and circle vertices, and check lab obstacles.
Geometric primitives	Included	GL_QUADS draws filled shapes; GL_LINES draws the grid/field lines; GL_LINE_LOOP draws outlines; GL_TRIANGLE_FAN draws circles.
Coordinates and vertices	Included	2D coordinates position scene objects; vertex coordinates form shapes and the lab grid.
Colors	Included	Colors distinguish floors, furniture, field markings, objects, and text panels.
GLUT callbacks	Included	glutDisplayFunc, glutKeyboardFunc, glutMouseFunc, and glutTimerFunc connect rendering, input, and timed updates.
On-screen text	Included	The opening title, controls/prompts, dialogue, name prompts, score, win/loss text, and closing message are drawn on screen.

Requirement	Status	Included in this project
Arrays	Included	gridVertices stores the repeated lab grid coordinates; labObstacles stores rectangles checked during movement.
Vertex arrays	Included	gridVertices is submitted through glVertexPointer() and rendered by glDrawArrays(GL_LINES, ...).

Module 1 connection: the program uses OpenGL for drawing and GLUT for window setup and events. Module 2 connection: the objects are built from primitive modes and vertex coordinates, with colors as visual attributes. Module 3 connection: the program draws raster-positioned bitmap text and responds through keyboard, mouse, display, and timer callbacks. Module 4 connection: the lab grid is stored in a vertex array and drawn as a batch.

The laboratory design uses a tiled floor, shelves, a table, Professor Oak, and a player sprite. The match design uses a green field, goals, a ball, Pokémon sprites, and a score display.

Program Flow

The program stores its current scene in the `GameState` variable. The `display()` function checks that state and draws the corresponding screen. Keyboard and mouse callbacks respond to user input and change the state when the player advances dialogue, enters names, chooses a Pokémon, reaches the exit, or starts the match. During the match, `updateField()` runs repeatedly using a GLUT timer to move the ball and Arceus, check collisions, update scores, and detect the end of the game.

Explanation of Important Functions

- `main()` initializes GLUT, creates the window, loads textures, sets up the 2D coordinate system, and registers the display, keyboard, mouse, and timer callbacks.
- `display()` draws the current screen based on the active game state.
- `drawText()` places bitmap text at a screen position.
- `drawRect()` draws a rectangle from four corner vertices.
- `drawSprite()` draws a full image texture; `drawSpriteFrame()` draws one frame from the player's sprite sheet.
- `drawLabTiles()` draws the laboratory floor and its vertex-array grid.
- `drawLabFurniture()` draws the shelves, table, Professor Oak, and, when appropriate, the Poké Balls.
- `blockedByFurniture()` checks whether the player's proposed position overlaps a laboratory obstacle.
- `mouse()` handles clicks on the Poké Balls and starter Pokémon.
- `keyboard()` handles names, dialogue, player movement, match actions, retry, and exit.
- `updateField()` updates the match repeatedly, including movement, collisions, scoring, and win or defeat conditions.
- `resetSoccerPositions()` resets the ball, player, and Arceus positions and the ball's velocity.
- `loadTexture()` loads image files and creates OpenGL textures.

4. Creativity Requirements

Visually distinguishable objects

Status: INCLUDED. The program shows more than five visually distinguishable objects: Professor Oak, the player character, the rival, three starter Pokémon, bookshelves, a table, Poké Balls, the soccer ball, Arceus, goals, and field markings.

At least three primitive types

Status: INCLUDED. Four primitive types are used: GL_QUADS for rectangles and sprite quads, GL_LINES for the tile grid and field line, GL_LINE_LOOP for outlines and the center circle, and GL_TRIANGLE_FAN for circular shapes such as the ball.

At least three application states

Status: INCLUDED. The GameState enumeration includes OPENING, PROF_DIALOGUE, NAME_PLAYER, NAME_RIVAL, FIRST_LAB, STARTER, SECOND_LAB, RIVAL_EVENT, FIELD, VICTORY, DEFEAT, and CLOSING.

On-screen text

Status: INCLUDED. Useful text appears as the title and start prompt, instructions, professor/rival dialogue, name-entry prompts, the match score, victory/defeat messages, and the closing instruction.

Keyboard interaction

Status: INCLUDED. WASD movement, Enter-based progression, name typing and deletion, R retry, and Escape/Enter exit behavior all have meaningful effects.

Mouse interaction

Status: INCLUDED. Mouse clicks select a Poké Ball to open the starter screen and select one of the starter Pokémon.

5. Creative Feature

Status: INCLUDED. The main creative feature is an interactive Pokémon-themed soccer match against a simple rival AI. The player can choose Bidoof, Magikarp, or Caterpie; each starter choice is accepted and appears in the match. Additional implemented features include:

- Animation: updateField() runs on a GLUT timer; the ball moves and slows, Arceus moves toward it, and the player walking sprite changes frame when moving in the lab.
- Scoring system: separate player and Arceus goal totals are displayed; the first side to three goals wins.
- Collision detection: the ball collides with the player and Arceus; lab movement is blocked by furniture obstacles.
- Character movement: WASD moves the player in both lab scenes and during the match; Arceus follows the ball.
- Multiple scenes: the story moves through dialogue, naming, both labs, starter selection, rival event, match, results, and closing. The first lab cannot be left until a Poké Ball is clicked.

6. Vertex Array Explanation Required by the Assignment

Required explanation point	Project-specific answer
What data is stored?	gridVertices stores the x/y coordinates of endpoints for the laboratory floor's vertical and horizontal grid lines. It has 232 float values representing 116 vertices.
Why use an array?	The grid is repeated visual data. Storing it together makes it easy to organize and reuse; one glDrawArrays call submits all grid vertices when the lab is drawn.
What does each vertex represent?	Each vertex is one endpoint of a grid line. Two vertices form each GL_LINES segment.
What primitive is rendered?	GL_LINES.
How is it configured?	createLabGrid() fills the array once. drawLabTiles() enables GL_VERTEX_ARRAY and calls glVertexPointer(2, GL_FLOAT, 0, gridVertices). Size 2 means x and y; GL_FLOAT is the data type; stride 0 means tightly packed coordinates. glDrawArrays(GL_LINES, 0, GRID_LINE_COUNT * 2) draws 116 vertices, forming 58 lines.
Why better than specifying repeatedly?	The coordinates are prepared once and organized in memory, then rendered together rather than issuing an individual vertex call for every grid endpoint on every frame.

Required explanation point	Project-specific answer
Indexed rendering?	The implementation uses glDrawArrays. It does not use glDrawElements() or an index array.

7. ChatGPT Use and Design Decisions

The assignment requires at least three examples of evaluating ChatGPT: suggestion, your decision, and reason. These examples are based on the project conversation; revise the wording if it does not match your own reasoning.

ChatGPT suggestion or advice	My decision	Reason
Use a separate verticalSpeed value to make up/down movement faster.	I removed the separate-speed change and kept the working movement calculation.	The change interfered with moving down, so I removed that recommendation.
Use a direct instruction telling the player exactly to go to the exit.	I chose the wording “Try moving around and find the exit.”	I wanted the prompt to be less direct and let the player explore.
Place the prompt near the exit; later advice suggested moving it to the upper-right to avoid covering the exit.	I placed the prompt at the bottom-left in the second lab.	I wanted it away from the exit while keeping it visible in the scene.
Use a while loop for ongoing game behavior.	The project uses conditions for decisions and a GLUT timer callback for repeated match updates.	A blocking while loop could prevent GLUT from processing input and redraw events.

8. Program Testing

The table records 17 functional checks. “Evidence attached” refers to the screenshots in Appendices A and B; “User-confirmed” records behavior you reported while running the program. The empty-name checks, lab controls, starter choices, match outcomes, and retry behavior are included.

#	Action/input	Expected result	Actual result	Status
1	Press Enter on opening screen	Professor dialogue appears	The opening screen and Professor dialogue are shown in Figures 1–2.	PASSED
2	Advance dialogue to player name entry	Player-name prompt appears	The player-name question and entry screen are shown in Figures 3–4.	PASSED
3	Press Enter while player-name field is empty	The empty name is not accepted; the program stays at name entry	You confirmed that Enter does not proceed until some text is entered.	PASSED
4	Type a player name and press Enter	Name is accepted and dialogue continues	Professor confirms ANDI in Figure 5.	PASSED
5	Press Enter while rival-name field is empty	The empty name is not accepted; the program stays at rival name entry	You confirmed that Enter does not proceed until some text is entered. Figure 6 shows the empty rival-name field.	PASSED
6	Type a rival name and press Enter	Name is accepted and dialogue continues	Professor confirms Berry in Figures 7–8.	PASSED

#	Action/input	Expected result	Actual result	Status
7	Move with W, A, S, D in the lab	Player moves in the corresponding directions	Player positions change across the lab screenshots in Figures 9–11.	PASSED
8	Move into a shelf or table obstacle	Player is blocked and does not pass through furniture	You confirmed that the obstacles work; Figure 12 shows the player at the table.	PASSED
9	Try to leave the first lab without choosing a Poké Ball	The player remains in the first lab	You confirmed that the first lab cannot be left until a Poké Ball is clicked.	PASSED
10	Click a Poké Ball and choose a starter	Starter screen opens; each of the three choices registers	Figure 13 shows the starter choices. You confirmed Bidoof, Magikarp, and Caterpie all register; Figure 26 shows Caterpie in the match.	PASSED
11	Reach the second-lab exit	Rival event appears	The rival event is shown in Figure 15 after the exit interaction.	PASSED
12	Press Enter in the rival event	Soccer match starts at 0–0	Figure 16 shows the match field and 0–0 score.	PASSED
13	Move into and kick the ball	Ball responds to player contact; scores update when it enters a goal	The match captures show the player and ball approaching in Figure 17, with score progression in Figures 18–19 and 22–23.	PASSED
14	Score three goals as the player	Victory state appears at three player goals	The 3-goal victory screen is shown in Figure 20.	PASSED
15	Allow Arceus to score three goals, then press Enter	Defeat state appears; Enter exits immediately from defeat	The defeat screen is shown in Figure 24. You confirmed Enter exits immediately from this state.	PASSED
16	Press R in defeat state	Match restarts with scores and player position reset	Figure 25 shows the 0–0 restart and Bidoof at the respawn position; you confirmed R works.	PASSED
17	Continue after victory and use closing controls	Closing screen appears; Enter or Escape exits from closing	The closing screen is shown in Figure 21. The final exit action itself is not captured.	PASSED

9. Screenshot Requirements

Appendices A and B contain screenshots from the story and laboratory, followed by the soccer field, score progression, victory, defeat, retry, Caterpie match, and closing screen.

Required evidence	Screenshot/caption
Initial screen	Appendix A, Figure 1
Main application	Appendix A, Figures 9–15; Appendix B, Figures 16–26
Keyboard interaction	Appendix A, Figures 4–5 and 9–12
Mouse interaction	Appendix A, Figure 13; Appendix B, Figure 26
Different application states	Appendix A, Figures 2–15; Appendix B, Figures 16–26
Creative feature	Appendix B, Figures 16–26 show the match, score changes, victory, defeat, retry, and starter choices in play.
Boundary/error test	Appendix A, Figure 12 shows the table obstacle test.
Final output	Appendix B, Figures 20–21 and 24 show victory, closing, and defeat.

Explanation of Conditions

Conditions select the correct drawing and behavior for each game state. For example, `display()` uses `if` and `else if` checks to decide whether to draw the opening, laboratory, starter screen, match, or result screen. In `keyboard()`, conditions check which key was pressed and whether the player is in a state where that key has an effect.

Other conditions enforce boundaries and game rules. Name entry advances only when the input is not empty. Laboratory movement is rejected when it would overlap furniture. A goal is counted only when the ball reaches the goal opening. The match ends when a score reaches three.

Explanation of Loops

Loops handle repeated data and repeated updates. `createLabGrid()` uses loops to generate the laboratory's vertical and horizontal grid lines. `drawBookshelf()` loops through positions to draw repeated book spines. The circle-drawing functions loop through angles to calculate vertices around a circle. `blockedByFurniture()` loops through the obstacle list and checks each rectangle.

The match does not use a blocking loop for continuous play. Instead, GLUT calls `updateField()` on a timer, allowing the application to keep processing input and redraws.

Keyboard Callback

The `keyboard()` function is registered with `glutKeyboardFunc()`. It receives a key press and checks the current game state. It handles WASD movement, text entry and Backspace, Enter-based progression, the R retry control after defeat, and Escape to exit. It also prevents an empty player or rival name from being confirmed.

Mouse Callback

The `mouse()` function is registered with `glutMouseFunc()`. It responds to a left-button click. It converts the mouse's vertical coordinate to match the program's OpenGL coordinate system, then checks whether the click falls inside a Poké Ball or starter Pokémon selection area. A valid click changes the game state and stores the selected starter.

Text Rendering

The `drawText()` function sets a color and raster position, then uses `glutBitmapCharacter()` to draw each character. It stops when it reaches the end of the text string. The program uses this function for dialogue, prompts, instructions, scores, and result messages.

Important Code Sections

The main code sections to show during the presentation are:

- `main()` for OpenGL and GLUT initialization and callback registration.
- `display()` for drawing each application state.
- `createLabGrid()` and `drawLabTiles()` for the vertex-array grid.
- `keyboard()` and `mouse()` for user input.
- `blockedByFurniture()` for laboratory obstacle collision.
- `updateField()` and `resolveBallVsRect()` for match movement, ball collisions, and scoring.
- `resetSoccerPositions()` for retry and kickoff reset behavior.

Errors and Problems Encountered

While checking the application, I paid attention to whether Enter could advance from the name screens without a name, whether the first laboratory could be left without selecting a Poké Ball, whether laboratory furniture blocked movement, and whether all three starter choices registered. I also checked the match's scoring, defeat, and retry behavior.

How I Solved the Problems

The name-entry condition checks that `nameInput` is not empty before accepting Enter. The first laboratory changes to the starter screen only when the mouse click falls within a Poké Ball's selection area. Starter selection uses separate click areas for Bidoof, Magikarp, and Caterpie. Laboratory movement checks the player's proposed position against the obstacle rectangles. In the match, R resets the scores and positions after defeat, while Enter exits from the defeat state.

How ChatGPT Was Used

I used ChatGPT to discuss project ideas, clarify graphics and interaction concepts, and help organize the project documentation. I compared suggestions with my intended design and the behavior I observed while running the program. I tested the controls and features myself and made the final decisions about what to keep or change.

What I Learned

I learned how to organize an interactive application into states, draw shapes with OpenGL primitives, place image textures, and use callbacks to handle keyboard and mouse input. I also learned how arrays can store repeated vertex data, how collision checks can prevent objects from overlapping, and how a timer callback can update a moving game scene.

Conclusion

Pokémon Field combines a short story, laboratory exploration, starter selection, and a soccer match in one interactive graphics project. Building it helped me apply OpenGL drawing, textures, text, callbacks, arrays, collision checks, and game-state logic. The screenshots and test cases document the project's main scenes and behaviors.

Reflection

What was the most difficult part?
The most difficult part was making the different game states and controls work together. The player needs to move through the labs, choose a starter, trigger the rival event, and then play the match.

What bug did you encounter?
I found that the name-entry screens needed to stay active when Enter was pressed without any text entered.

How did you fix it?
I added a check so Enter confirms the player or rival name only when the name field is not empty. I also made the first lab wait for a Poké Ball click before opening starter selection.

What did ChatGPT help you understand?
ChatGPT helped me understand how the lab grid's vertex data is passed to OpenGL using `glVertexPointer()` and drawn with `glDrawArrays()`. It also helped me describe how the keyboard and mouse callbacks respond to input.

What did ChatGPT suggest that you changed?
ChatGPT suggested adding a separate vertical movement speed. I removed that change because it interfered with moving down, and kept the movement calculation that worked. I also changed the lab prompt to "Try moving around and find the exit" so it would encourage exploration.

What part of the program did you personally improve?
I improved the interaction flow and checked that the obstacle blocking and all three starter choices worked. I also made sure the first lab could not be left before a Poké Ball was clicked.

If you had another week, what feature would you add?
I would add sound effects and more animations, such as a sound when the ball is kicked or a short animation when a goal is scored.

Appendix A Story and Laboratory Screenshots

The in-game names shown in these captures are ANDI and Berry.

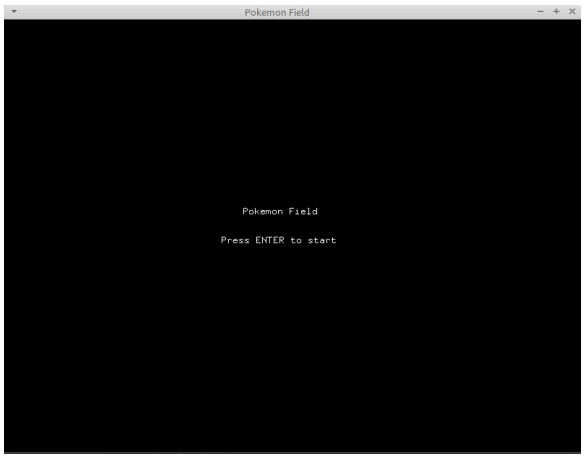


Figure 1. Opening screen with the ENTER start prompt.



Figure 2. Professor Oak introduction dialogue.



Figure 3. Professor asks for the player name.

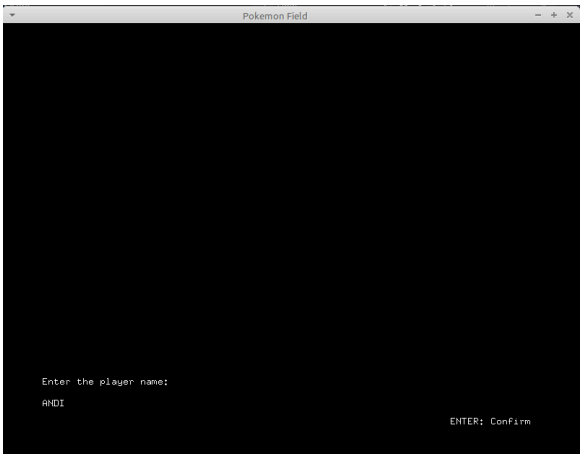


Figure 4. Player name entry screen with ANDI typed.

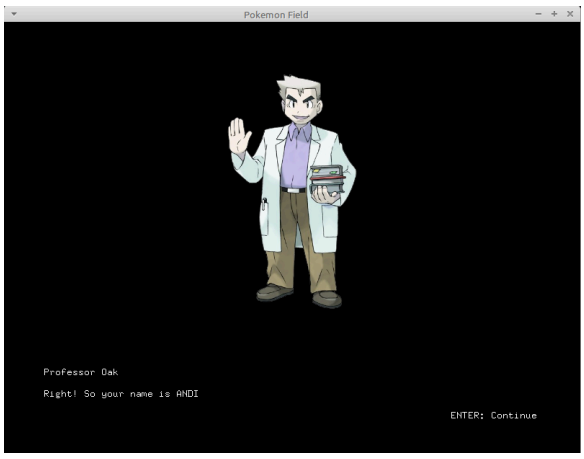


Figure 5. Professor confirms the player name ANDI.

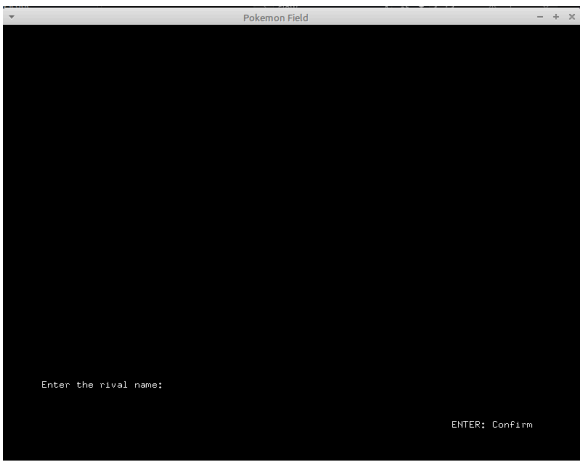


Figure 6. Rival name entry screen with the field empty.

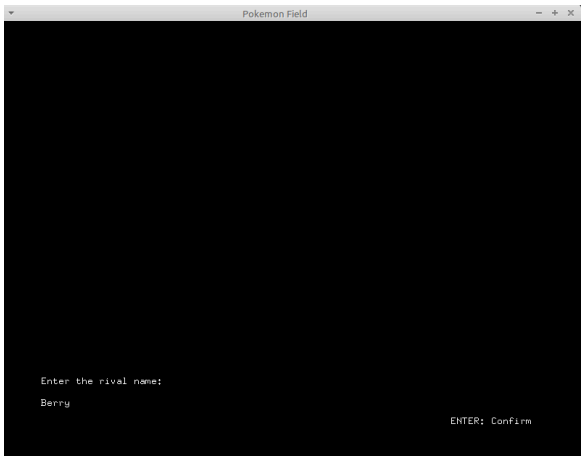


Figure 7. Rival name entry screen with Berry typed.

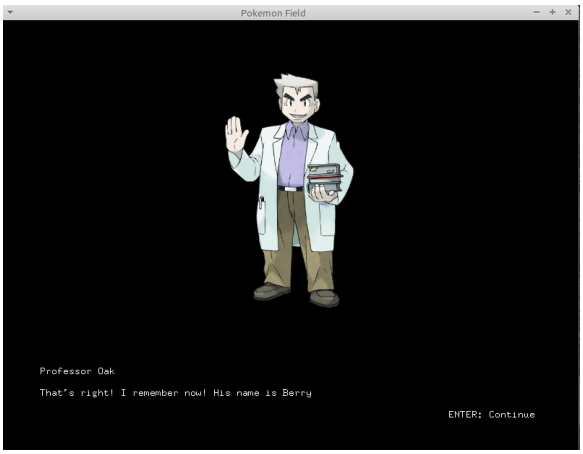


Figure 8. Professor confirms the rival name Berry.

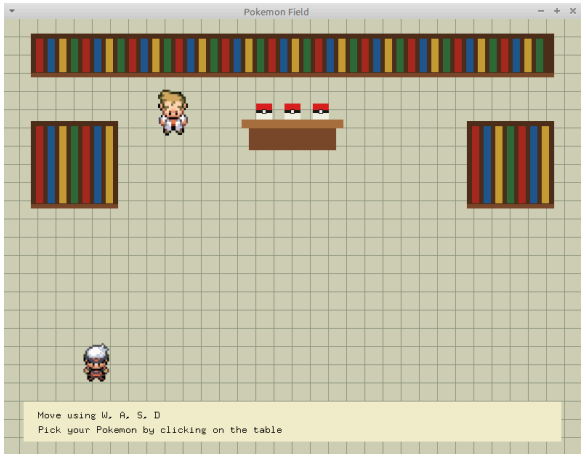


Figure 9. First laboratory with player at starting position.

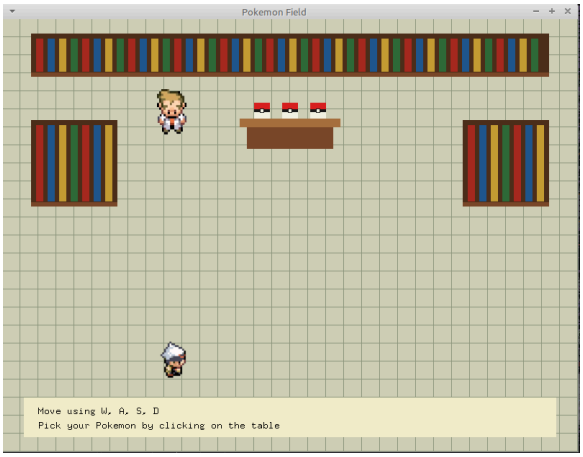


Figure 10. Player moved with WASD in the first laboratory.

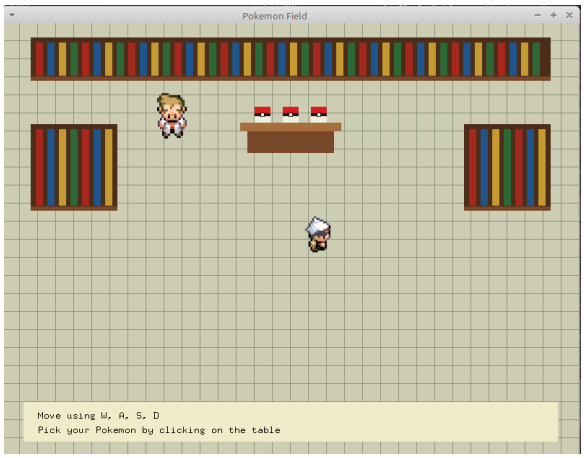


Figure 11. Player at another position in the laboratory.

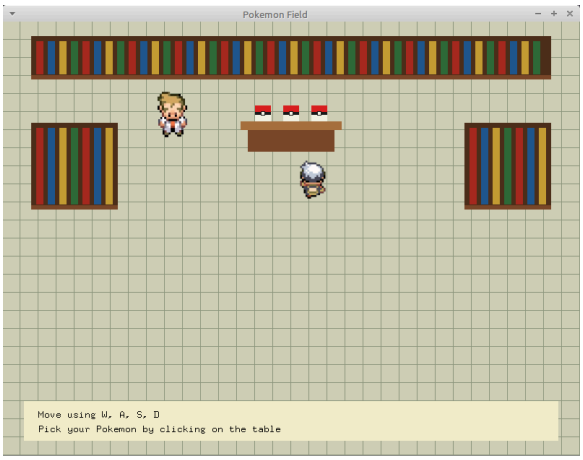


Figure 12. Player stopped by the laboratory table obstacle.



Figure 13. Starter selection screen opened by clicking a Poké Ball.

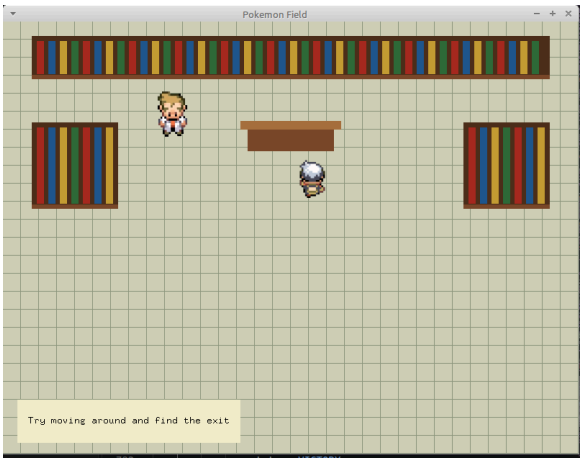


Figure 14. Second laboratory with the exit prompt.



Figure 15. Rival event after reaching the exit.

Appendix B Soccer Match Screenshots

These figures document the match kickoff, player and rival scoring, both outcomes, restart, and the Caterpie selection.

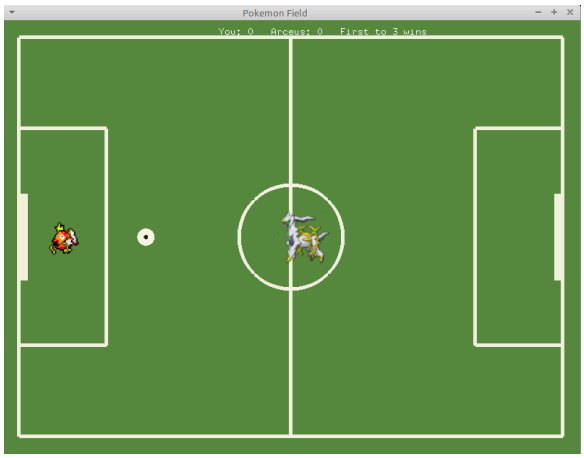


Figure 16. Soccer match begins with both scores at zero.

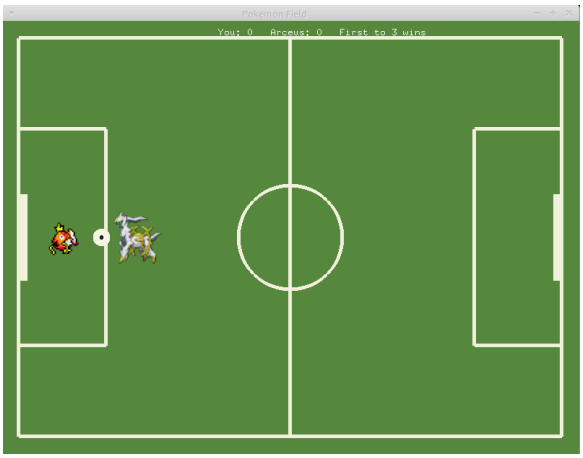


Figure 17. Player and ball positioned near the left goal.

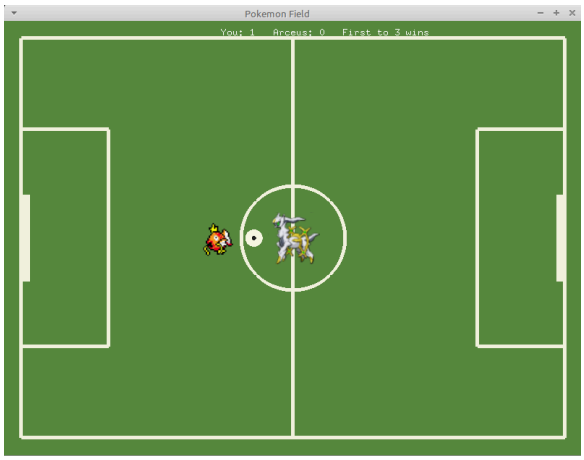


Figure 18. Player score increases to one.

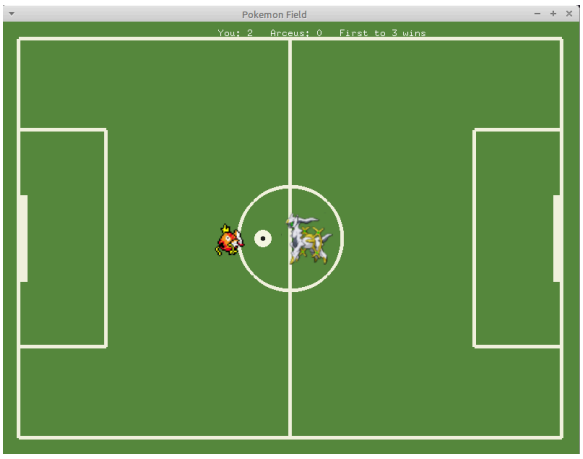


Figure 19. Player score increases to two.

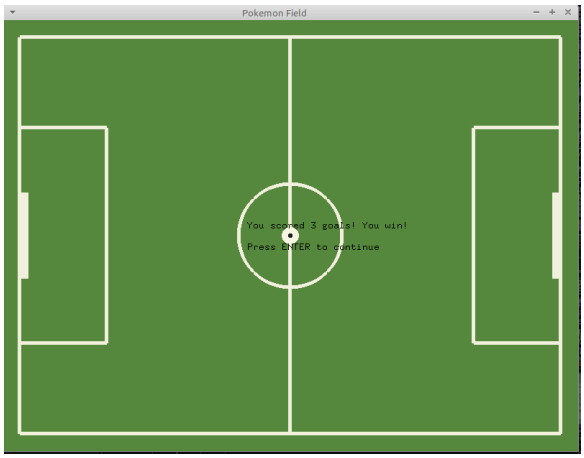


Figure 20. Victory after the player reaches three goals.

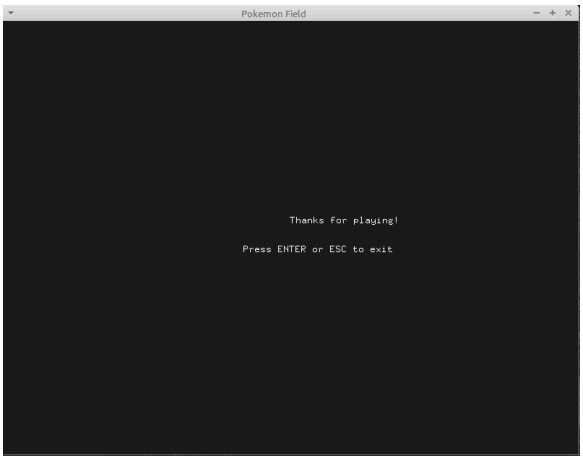


Figure 21. Closing screen after the match.

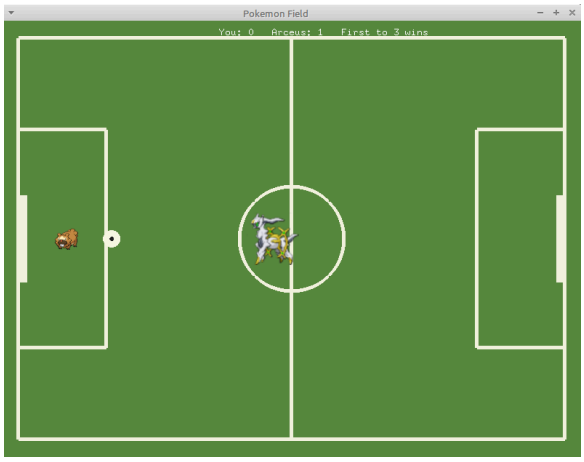


Figure 22. Arceus score increases to one.

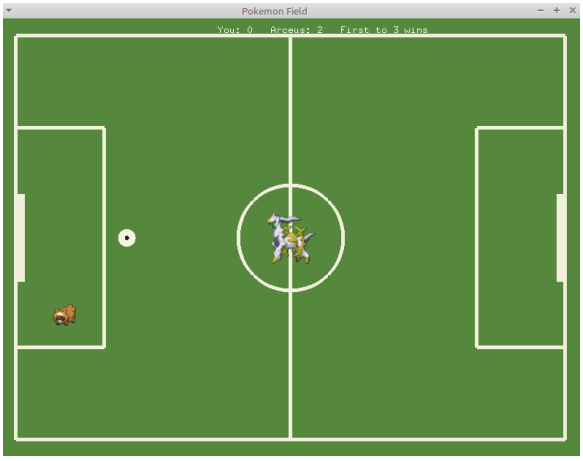


Figure 23. Arceus score increases to two.



Figure 24. Defeat screen after Arceus reaches three goals.

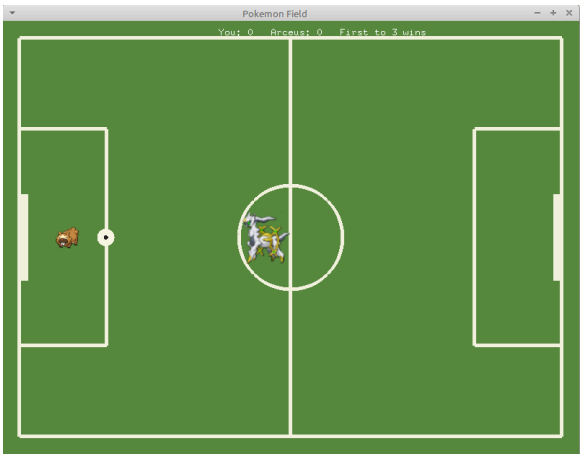


Figure 25. Retry returns to 0–0 with Bidoof at the reset position.

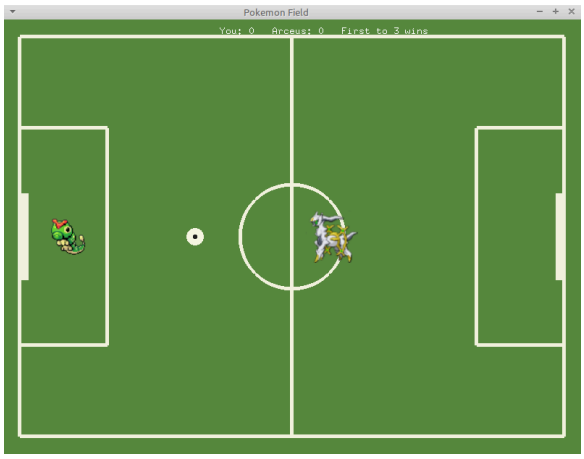


Figure 26. Caterpie selected and visible in the match at 0–0.